



CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

NODE.JS E TYPESCRIPT

ATIVIDADE DE FIXAÇÃO

NÍVEA PINTO DE SOUZA

JOINVILLE/SC

2026

Sumário

PESQUISA E FUNDAMENTAÇÃO	3
1. Event loop do Node.js	4
2. Operações síncronas e assíncronas	5
3. TypeScript e tipagem	5
4. CommonJS e ES Modules	5
5. Express, Fastify e NestJS	6
6. nvm e gerenciamento de versões	6
7. Container Docker e máquina virtual	6
Referências	7

PESQUISA E FUNDAMENTAÇÃO

Objetivo: apresentar, de forma fundamentada e com exemplos próprios, conceitos de Node.js, TypeScript, frameworks web, gerenciamento de versões e containers.

1. Event loop do Node.js

O event loop é o mecanismo que organiza a execução das tarefas em uma aplicação Node.js. O código JavaScript normalmente é executado em uma thread principal, mas isso não significa que o servidor precise esperar cada operação terminar antes de atender outra requisição. Operações de entrada e saída, como consultas a banco, acesso à rede e leitura de arquivos, podem ser iniciadas de forma não bloqueante. Enquanto elas aguardam uma resposta, o event loop continua processando outros eventos e, quando a operação termina, o callback ou a continuação da Promise volta para a fila de execução. Por isso um servidor Node.js consegue manter muitas conexões simultâneas sem criar uma thread JavaScript principal para cada cliente. Um exemplo seria uma API que recebe cem requisições que consultam um banco: ela pode iniciar essas consultas e continuar recebendo trabalho enquanto espera as respostas, em vez de ficar parada em cada consulta.

2. Operações síncronas e assíncronas

Uma operação síncrona mantém o fluxo esperando sua conclusão antes de avançar. Já uma operação assíncrona permite iniciar uma tarefa e continuar outras atividades enquanto o resultado não está disponível. Em Node.js isso é especialmente importante porque os callbacks JavaScript passam pelo event loop. Se uma rota executar um cálculo pesado ou uma operação síncrona demorada por vários segundos, outras requisições podem ficar esperando. Em produção isso é grave porque um único processamento ruim pode aumentar a latência de muitos usuários e até facilitar indisponibilidade do serviço. Por exemplo, seria inadequado uma API ler um arquivo enorme de forma síncrona a cada requisição; uma alternativa é usar APIs assíncronas ou deslocar processamento realmente pesado para workers.

3. TypeScript e tipagem

TypeScript é uma linguagem que amplia JavaScript adicionando, entre outros recursos, um sistema de tipos verificado antes da execução. No JavaScript puro, uma variável pode receber valores de tipos diferentes e vários erros só aparecem quando determinada linha realmente é executada. TypeScript realiza verificação estática durante o desenvolvimento/compilação e consegue avisar, por exemplo, que uma função esperava um número mas recebeu uma string. Isso não transforma JavaScript em uma linguagem que valida tipos automaticamente durante a execução: depois de compilado, os tipos do TypeScript não funcionam como uma camada de validação em runtime. Para dados externos, ainda é necessário validar a entrada. A vantagem é detectar antecipadamente uma classe grande de inconsistências e tornar contratos entre funções e módulos mais explícitos.

4. CommonJS e ES Modules

CommonJS é o sistema de módulos historicamente associado ao Node.js e utiliza `require()` para importar e `module.exports/exports` para exportar. ES Modules é o padrão oficial do ECMAScript e usa `import` e `export`. Hoje o Node.js suporta os dois modelos, porém a forma como um arquivo é interpretado depende de fatores como extensão (`.cjs` ou `.mjs`) e o campo `type` do `package.json`. Isso ainda causa incompatibilidades porque bibliotecas e projetos podem adotar sistemas diferentes, e detalhes de carregamento e interoperabilidade não são idênticos. Assim, copiar um exemplo com `require` para um projeto configurado como ESM, ou importar um pacote com expectativas diferentes sobre `exports`, pode gerar erros. Em equipes, definir explicitamente o padrão do projeto reduz esse tipo de problema.

5. Express, Fastify e NestJS

Express é uma escolha apropriada quando a equipe deseja um framework pequeno, flexível e com poucas decisões impostas. Ele fornece roteamento e middleware e deixa a arquitetura principalmente a cargo dos desenvolvedores. Eu o escolheria para uma API pequena ou média, como um serviço interno de cadastro de equipamentos, em que a equipe já possui um padrão de pastas e quer montar somente os componentes necessários.

Fastify também é voltado a aplicações web em Node.js, mas enfatiza baixo overhead, desempenho, plugins e uso de schemas para validação/serialização. Ele seria uma boa opção para uma API que recebe grande volume de chamadas, como um serviço de coleta de eventos, desde que a equipe meça o desempenho no cenário real. Além da performance, sua arquitetura de plugins e encapsulamento ajuda a organizar funcionalidades reutilizáveis.

NestJS oferece uma estrutura mais opinativa e arquitetural, com módulos, controllers, providers, injeção de dependência e suporte amplo a recursos como validação, filas, microsserviços e documentação de APIs. Eu o escolheria para um backend empresarial maior, desenvolvido por várias pessoas e que deve crescer por bastante tempo. A padronização inicial pode exigir mais conceitos, porém facilita manter responsabilidades separadas e uma organização consistente.

6. nvm e gerenciamento de versões

Um gerenciador de versões como o nvm permite instalar e alternar entre diferentes versões do Node.js no mesmo computador. Isso é importante porque projetos diferentes podem depender de versões distintas do runtime. Em um time, padronizar a versão reduz situações em que o projeto funciona para uma pessoa e falha para outra por diferenças de ambiente. Por exemplo, um sistema legado pode precisar de uma versão anterior enquanto um projeto novo utiliza uma versão atual. Com um gerenciador de versões, o desenvolvedor troca de runtime conforme o projeto sem reinstalar manualmente o Node a cada vez. Arquivos como .nvmrc também podem registrar a versão esperada pelo projeto.

7. Container Docker e máquina virtual

Um container empacota a aplicação e suas dependências em um ambiente isolado de execução. Conceitualmente, containers compartilham o kernel do sistema operacional hospedeiro, enquanto uma máquina virtual virtualiza hardware e normalmente executa um sistema operacional convidado completo. Por isso containers tendem a iniciar mais rapidamente e consumir menos recursos que VMs para muitos cenários de aplicação. Uma VM continua sendo útil quando é necessário um isolamento em nível de sistema operacional ou executar um sistema convidado próprio. Em desenvolvimento, um container pode executar PostgreSQL com uma versão definida sem exigir que cada integrante instale e configure o banco diretamente na máquina.

Referências

NODE.JS. Don't Block the Event Loop (or the Worker Pool). Node.js Documentation. Disponível em: <https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop>. Acesso em: 28 ago. 2026.

EXPRESS.JS. Express - Node.js web application framework. Disponível em: <https://expressjs.com/>. Acesso em: 28 ago. 2026.

DOCKER. What is a Container? Disponível em: <https://www.docker.com/resources/what-container/>. Acesso em: 28 ago. 2026.